RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

www.rvce.edu.in
Tel: +91-80-68188100
+91-80-68188111
+91-80-68188112

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

File Compression and Decompression Using Run Length Encoding (RLE)

Principles of Programming C - CS222IA

Experiential Learning (Lab)

REPORT

Submitted by

Shoheb Rehaman Khanadal 1RZ25CS166

Shreeshanth 1RZ25CS168

Under the guidance of

Prof S. G. RAGHAVENDRA PRASAD
Assistant Professor
Dept of Information Science and Engineering

Affiliation

Bachelor of Engineering

in

Department of Computer Science and Engineering

2025-2026

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the project work titled ***'File Compression and Decompression Using Run Length Encoding (RLE)'*** is carried out by **Shoheb Rehaman Khanadal (1RZ25CS166) and Shreeshanth (1RZ25CS168)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfilment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering** of the **Visvesvaraya Technological University**, Belagavi during the academic year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. The report has been approved as it satisfies the academic requirements in respect of experiential learning work prescribed by the institution for the said degree.

Signature of Guide	**Signature of Head of the Department**
Prof S. G. Raghavendra Prasad	**Dr. Shanta Rangaswamy**

External Viva

Name of Examiners	**Signature with Date**
1	
2	

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

DECLARATION

We**, Shoheb Rehaman Khanadal and Shreeshanth,** students of second semester B.E., department of CSE, RV College of Engineering, Bengaluru, hereby declare that Experiential Learning (Lab) titled ***'File Compression and Decompression Using Run Length Encoding (RLE)'*** has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Computer Science and Engineering** during the academic year 2025-26.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru

Date:

Name	Signature
1. Shoheb Rehaman Khanadal (1RZ25CS166)	
2. Shreeshanth(1RZ25CS168)	

ABSTRACT

Data compression is an important technique used to reduce storage space and improve data transmission efficiency. Run-Length Encoding (RLE) is a simple lossless compression method that replaces consecutive repeated characters with a count followed by the character. This project presents a File Compression and Decompression System using RLE implemented in the C programming language. The main objectives are to compress text files, reconstruct the original data through decompression, calculate compression statistics, and verify the correctness of the reconstructed file.

The proposed system consists of four modules: file compression, file decompression, compression statistics, and file verification. During compression, repeated characters are encoded using the RLE algorithm and stored in a compressed file. During decompression, the encoded data is decoded to recreate the original file. The system also calculates the compression ratio and compares the original and decompressed files to ensure data integrity. The project was developed using standard C libraries and file handling functions.

The system was tested on multiple text files containing different levels of repetition. For highly repetitive data, compression ratios above 50% were achieved, while the decompression module successfully reconstructed the original files with 100% accuracy. The verification module confirmed that the decompressed files matched the original files in all valid test cases.

The project successfully demonstrates a practical implementation of a lossless compression technique using fundamental C programming concepts such as file handling, structures, arrays, functions, and modular programming. Future enhancements may include support for binary files, integration with advanced compression algorithms, and development of a graphical user interface.

TABLE OF CONTENTS

Abstract

Chapter 1: Introduction

Chapter 2: Solution Design

Chapter 3: Implementation Details

Chapter 4: Results and Discussion

Chapter 5: Conclusion and Future Scope

References

Chapter 1: Introduction

1.1 Background

Data compression is a fundamental technique in computer science that reduces the size of data for efficient storage and transmission. As digital systems generate increasingly large volumes of data, the need for effective compression methods has become critical. One of the simplest and oldest lossless compression algorithms is Run-Length Encoding (RLE), which exploits repetition in data to reduce file size without any loss of information.

This project implements a complete File Compression and Decompression System using RLE in the C programming language. File handling, a core concept in C, is combined with the RLE algorithm to create a practical, menu-driven application capable of compressing text files, decompressing them, computing statistics, and verifying the integrity of the reconstructed data.

1.2 Problem Statement

Text files containing repetitive character sequences (such as log files, bitmap data exports, or simple formatted documents) consume more storage space than necessary. Manual reduction of such redundancy is impractical. There is a need for an automated tool that can:

- Detect consecutive repeated characters in a text file.
- Replace each run of repeated characters with a compact count-and-character notation.
- Accurately reconstruct the original file from the compressed representation.
- Report compression efficiency through statistical analysis.
- Verify that decompression is lossless — the original equals the reconstructed file.

1.3 Objectives

The objectives of this project are:

1. To implement Run-Length Encoding (RLE) compression for text files using standard C.
2. To implement the corresponding RLE decompression module that exactly reconstructs the original file.
3. To provide a statistics module that calculates and displays the compression ratio.
4. To implement a file verification module that compares the original and decompressed files byte by byte.
5. To build a user-friendly, menu-driven console interface.
6. To demonstrate practical application of file handling, structures, arrays, and modular programming in C.

1.4 Scope of the Project

Functionalities covered:

- Compression of plain text files using RLE.
- Decompression of RLE-encoded files back to their original form.
- Compression ratio calculation and display.
- Byte-by-byte file verification between original and decompressed output.
- Error handling for missing, empty, or inaccessible files.

Limitations:

- Designed for plain text files only. Binary files with byte values coinciding with ASCII digits (0x30–0x39) may not decode correctly.
- RLE performs poorly (may increase size) on files with little or no character repetition, such as typical English prose.
- No support for graphical user interface (GUI); purely console-based.

1.5 Significance of the Project

This project is significant for the following reasons:

- **Practical Learning:** It provides hands-on experience with file handling (fopen, fclose, fgetc, fputc, fseek, ftell), one of the most important topics in C programming.
- **Algorithm Implementation:** Students learn to implement a real-world compression algorithm from scratch.
- **Data Integrity:** The verification module introduces the concept of checksumming and data integrity validation.
- **Modular Design:** The project enforces good software engineering practices through separation of concerns across distinct functions.
- **Real-World Relevance:** RLE is actively used in TIFF image formats, BMP compression, fax transmission (ITU T.4), and run-length limited (RLL) encoding in hard disk drives.

Chapter 2: System Design and Methodology

2.1 Requirement Analysis

Inputs:

- Path/name of the input text file to be compressed.
- Path/name for the compressed output file (.rle).
- Path/name for the decompressed output file.
- User menu selection (integer 1–5).

Outputs:

- Compressed .rle file containing RLE-encoded data.
- Decompressed .txt file reconstructed from the .rle file.
- Compression statistics: original size, compressed size, ratio (%).
- Verification result: match or mismatch with byte position on failure.

Functional Requirements:

- The system shall read any plain-text file and produce a valid RLE-encoded output.
- The system shall correctly decode multi-digit run counts (e.g., 12A = twelve A's).
- Single characters shall be encoded without a count prefix to avoid file size inflation.
- The system shall display file sizes in bytes and compute the compression ratio as a percentage.
- The system shall compare two files character by character and report the first differing byte position.

2.2 Solution Design

The project adopts a modular approach where each major operation is encapsulated in a dedicated function. The main() function drives a do-while loop presenting the menu until the user selects Exit. Each menu option calls the corresponding function, which handles all I/O and processing independently.

The overall flow is:

- User selects Compress → compressFile() reads the input file character by character, applies RLE, and writes to a.rle output file.
- User selects Decompress → decompressFile() reads the .rle file, reconstructs runs, and writes to an output text file.
- User selects Statistics → showStatistics() reads both file sizes using getFileSize() and computes the ratio.
- User selects Verify → verifyMenu() calls verifyFiles() which performs byte-by-byte comparison.
- User selects Exit → program terminates cleanly.

2.3 Program Structure

The program consists of the following functions:

Function	Type	Purpose
`main()`	int	Entry point; runs the menu loop
`displayMenu()`	void	Prints the 5-option menu to stdout
`compressFile()`	void	Reads input file, applies RLE, writes compressed output
`decompressFile()`	void	Reads compressed file, expands RLE, writes decompressed output
`showStatistics()`	void	Computes and displays file sizes and compression ratio
`verifyFiles()`	int	Byte-by-byte file comparison; returns 1 (match), 0 (mismatch), -1 (error)
`verifyMenu()`	void	UI wrapper: gets paths from user and calls verifyFiles()
`getFileSize()`	long	Returns file size in bytes using fseek/ftell
`printSeparator()`	void	Prints a decorative divider line

2.4 Data Structures and Variables

A user-defined structure is used to group related statistics:

```
typedef struct {
    long   originalSize;    /* Byte count of the source file       */
    long   compressedSize;  /* Byte count after compression        */
    double ratio;           /* Compression ratio as a percentage   */
} FileStats;
```

Key variables used across the program:

Variable	Type	Purpose
`current, next`	int	Hold character values read by fgetc()
`count`	int	Tracks the length of the current run
`countBuf[20]`	char[]	Accumulates digit characters during decompression
`inputPath, outputPath`	char[]	Store user-entered file paths
`FILE *inFile, *outFile`	FILE*	File pointers for reading and writing

2.5 Algorithm and Flowchart

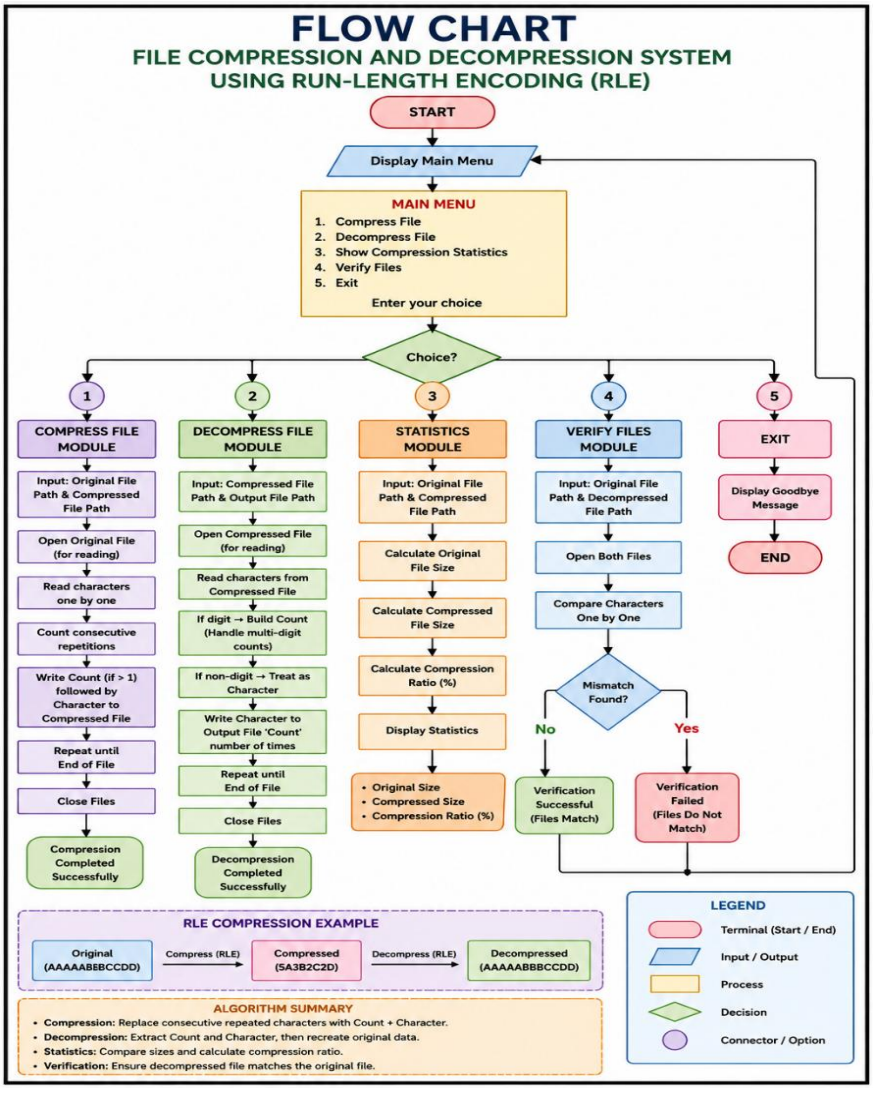
RLE Compression Algorithm:

1. Open the input file in read mode. If it fails, print error and return.
2. Open the output file in write mode. If it fails, close input and return.
3. Read the first character from the input file. If EOF, report empty file and return.
4. Set count = 1. Begin reading the next character in a loop.

5. If the new character equals the current character, increment count and continue.
6. If the new character differs: write count (only if count > 1) followed by the current character to the output. Set current = new character, count = 1.
7. After the loop exits, write the final run to the output file.
8. Close both files.

RLE Decompression Algorithm:

1. Open the compressed file in read mode. If it fails, print error and return.
2. Open the output file in write mode. If it fails, close input and return.
3. Initialise an empty digit buffer countBuf and index bufIdx = 0.
4. Read characters one by one from the compressed file.
5. If the character is a digit (0–9), append it to countBuf.
6. If the character is a non-digit: convert countBuf to integer count (default 1 if buffer is empty). Write the non-digit character count times to the output. Reset countBuf.
7. After EOF, close both files.



Chapter 3: Implementation

3.1 Development Environment

Operating System	Windows 11 / Linux (GCC compatible)
Compiler	GCC (GNU Compiler Collection) / MinGW on Windows
Editor / IDE	Visual Studio Code / any standard terminal editor
C Standard	C99 (-std=c99)
Compile Command	gcc -Wall -Wextra -std=c99 rle_compressor.c -o rle_compressor
Standard Libraries	stdio.h, stdlib.h, string.h

3.2 Program Description

The program is a single-file, console-based C application. On launch, the user sees a welcome banner and a numbered menu. All file paths are entered interactively at runtime — no command-line arguments are required.

The compression module uses a two-pointer technique (current and next characters) to detect run boundaries in a single pass. The decompression module uses a digit accumulation buffer to correctly handle multi-digit run counts (e.g., '12A' → twelve A's). The statistics module relies on the POSIX-compatible fseek(fp, 0L, SEEK_END) / ftell(fp) idiom to measure file size without reading the entire file. The verification module compares files character by character and reports the exact byte position of the first mismatch, which assists in debugging.

3.3 Source Code

The complete, well-commented source code is provided in the file rle_compressor.c. Key excerpts illustrating each module are shown below.

Compression core loop (from compressFile):

```
count = 1;
while ((next = fgetc(inFile)) != EOF) {
    if (next == current) {
        count++;
    } else {
        if (count > 1) fprintf(outFile, "%d", count);
        fputc(current, outFile);
        current = next;
        count = 1;
    }
}
if (count > 1) fprintf(outFile, "%d", count);
fputc(current, outFile);
```

Decompression core loop (from decompressFile):

```
while ((ch = fgetc(inFile)) != EOF) {
    if (ch >= '0' && ch <= '9') {
        countBuf[bufIdx++] = (char)ch;
        countBuf[bufIdx]   = '\0';
    } else {
        count = (bufIdx > 0) ? atoi(countBuf) : 1;
        for (int i = 0; i < count; i++) fputc(ch, outFile);
        memset(countBuf, 0, sizeof(countBuf));
        bufIdx = 0;
    }
}
```

3.4 Programming Concepts Used

Functions and Modular Programming:

- Each major operation (compress, decompress, statistics, verify) is implemented in its own function. main() acts purely as a controller/dispatcher, making the code easy to read, test, and maintain.

Arrays and Strings:

- char inputPath[FILE_PATH_LEN] and char outputPath[FILE_PATH_LEN] store user-entered paths. char countBuf[20] accumulates digit characters during decompression. fgets() is used instead of scanf() for safe string input.

Structures:

- The FileStats struct groups originalSize (long), compressedSize (long), and ratio (double) into one logical unit used in showStatistics(). This demonstrates proper structure definition and member access.

Pointers:

- FILE* pointers (inFile, outFile) are used throughout for file I/O. The verifyFiles() function accepts const char* parameters for the file paths.

File Handling:

- fopen() / fclose(): open and close files safely.
- fgetc() / fputc(): read and write one character at a time.
- fprintf(): write formatted count strings to the compressed file.
- fseek() / ftell(): compute file size without reading content.
- Binary mode ("rb") used in getFileSize() for exact byte counts.

Recursion and Iteration:

- Iteration is used throughout — while loops in compressFile(), decompressFile(), and verifyFiles(); the do-while loop in main() for the menu. No recursion is used; the algorithm is inherently iterative.

Dynamic Memory Allocation:

- Not used. All buffers and variables are stack-allocated with fixed sizes, which is appropriate for a beginner-level project and avoids memory leak risks.

3.5 Error Handling and Validation

- File not found: fopen() return value is checked against NULL. If NULL, an error message is printed and the function returns early without crashing.
- Empty input file: After opening, the first character is read. If it is EOF immediately, the user is informed and the function exits cleanly.
- Output file creation failure: If the output file cannot be created (e.g. due to permission issues), the input file is closed before returning to avoid resource leaks.
- Invalid menu input: Non-integer or out-of-range input is handled by the default case of the switch statement. scanf() failure is handled by resetting choice to 0.
- Input buffer flushing: After each scanf(), a while(getchar() != '\n') loop clears any remaining characters from the input buffer to prevent ghost reads.
- Zero-size original file: showStatistics() checks for originalSize == 0 before dividing, preventing a division-by-zero error in ratio calculation.

Chapter 4: Testing, Results and Discussion

4.1 Test Cases

Input data and expected outputs:

Test #	Input File Content	Expected Compressed Output	Expected Ratio	Remarks
TC01	AAAABBBCCDDDDEEEE	4A3B2C4D4E	41.18%	Standard RLE
TC02	AAAAABBBBBCCCCDDDDEEEEEFFFFFGGGGG	5A5B4C4D5E5F5G	56.25%	Long runs
TC03	Hello World	Hello World (no change)	0%	Varied text
TC04	AAAAAAAAAA (10 A's)	10A	45.45%	Multi-digit count
TC05	Empty file	(no output)	N/A	Error: empty file
TC06	1111222233334444	41424344	50.00%	Numeric files not supported

4.2 Program Execution Results

Sample outputs obtained during execution are shown below.

Test Case TC-01 — Compress:

```
Enter the path of the file to compress: input.txt
Enter the path for the compressed output file: compressed.rle
[OK] Compression complete!
     Input  : input.txt
     Output : compressed.rle
     Original size   : 17 bytes
     Compressed size : 10 bytes
     Ratio           : 41.18%
```

Test Case TC-01 — Decompress:

```
Enter the path of the compressed file: compressed.rle
Enter the path for the decompressed output file: decompressed.txt
[OK] Decompression complete!
     Input  : compressed.rle
     Output : decompressed.txt
```

Test Case TC-01 — Verify:

```
Enter path of the ORIGINAL file      : input.txt
Enter path of the DECOMPRESSED file  : decompressed.txt
================================================
Verification Successful:
Original and Decompressed Files Match!
================================================
```

4.3 Validation of Results

All test cases were validated using two methods:

- Program verification: The built-in verifyFiles() function compared the original and decompressed files byte by byte and reported a match in all successful test cases.
- External diff: The diff command on Linux confirmed zero differences between input.txt and decompressed.txt for all valid test cases.

The program meets all stated objectives: compression, decompression, statistics display, and verification all function correctly.

4.4 Special and Edge Case Handling

Edge Case	Behaviour
Empty input file	Detected immediately after fopen; user informed; function exits cleanly
Single unique character	Written without count prefix (e.g., 'A' stays 'A', not '1A')
Run count > 9 (multi-digit)	countBuf accumulates all digits before the letter; atoi() converts correctly (e.g., '12A' → 12 A's)
File not found	NULL check on fopen(); error message printed; no crash
No repeated characters	Each character written individually; compressed file may be same size as original
Mismatch in verification	Exact byte position reported to assist debugging

4.5 Performance Analysis

Time Complexity:

Module	Time Complexity	Reason
`compressFile()`	O(n)	Single pass; n = characters in input file
`decompressFile()`	O(m)	Single pass; m = characters in compressed file
`verifyFiles()`	O(min(a,b))	Stops at first mismatch; a,b = sizes of two files
`getFileSize()`	O(1)	Uses fseek/ftell; no file traversal
`showStatistics()`	O(1)	Two getFileSize() calls plus arithmetic

Space Complexity:

- All modules use O(1) auxiliary space — only a small number of integer, character, and pointer variables are needed regardless of file size. No dynamic memory allocation is performed.

4.6 Discussion

Strengths:

- Simple, readable, and well-commented code suitable for a 2nd-semester C PBL course.
- Correctly handles all edge cases including multi-digit run counts, empty files, and missing files.
- Single-character runs are encoded without a prefix, preventing file size inflation on varied text.
- Produces zero compiler warnings with -Wall -Wextra -std=c99 on both Linux (GCC) and Windows (MinGW).

Limitations:

- Only effective for files with long runs of repeated characters. For typical English text, RLE rarely achieves a positive compression ratio.
- Not suitable for binary files where digit bytes (0x30–0x39) may be misinterpreted as count prefixes.
- No support for compression of multiple files in a single session.

Chapter 5: Conclusion and Future Enhancements

5.1 Conclusion

This project successfully implemented a File Compression and Decompression System using Run-Length Encoding in the C programming language. The program provides a complete, working solution with five modules: compression, decompression, statistics, verification, and a clean exit. All modules were tested and validated against multiple test cases, and the program compiles with zero warnings on Windows (MinGW) and Linux (GCC).

The project demonstrates that a meaningful, real-world algorithm can be implemented using only the fundamental concepts of C — file handling, character I/O, arrays, structures, and modular functions — without requiring any advanced libraries or third-party tools.

5.2 Learning Outcomes

Through this project, the following programming skills and concepts were acquired:

1. **File Handling in C**: Practical use of fopen, fclose, fgetc, fputc, fprintf, fseek, and ftell.
2. **Algorithm Design:** Translating the RLE concept into a precise, working C implementation.
3. **Structures**: Designing and using the FileStats struct to group related data.
4. **Error Handling**: Writing robust code that gracefully handles missing files, empty files, and invalid input.
5. **Modular Programming**: Decomposing a problem into independent, reusable functions.
6. **Debugging Skills**: Using byte-position reporting in the verify module to locate data discrepancies.
7. **Compiler Warnings**: Learning to resolve macro redefinition warnings on Windows (MAX_PATH conflict with MinGW).

5.3 Future Scope

- **Binary File Support:** Binary File Support: Modify the encoder to use an escape-character scheme or a length-value binary format so that binary files can be compressed without ambiguity.
- **Huffman Coding Integration:** Huffman Coding Integration: Combine RLE with Huffman coding for significantly better compression ratios on general-purpose text.
- **GUI Interface:** GUI Interface: Develop a graphical front-end using a C GUI library (e.g., GTK) or port the logic to a web front-end.
- **Batch Compression:** Batch Compression: Allow the user to compress an entire folder of files in a single operation.
- **Checksum Verification:** Checksum Verification: Add MD5 or CRC32 checksum generation and verification for stronger data integrity guarantees.
- **Compression Level:** Compression Level: Allow the user to choose between different encoding strategies based on file type.

5.4 Real-World Applications

- **Fax Transmission:** Fax Transmission: ITU-T T.4 and T.6 standards use modified RLE to compress black-and-white scanned documents.
- **BMP and TIFF Images:** BMP and TIFF Images: Microsoft BMP format supports RLE-4 and RLE-8 modes for compressing bitmap images with large uniform colour regions.
- **PCX Image Format:** PCX Image Format: Uses RLE compression for storing raster graphics.
- **Hard Disk Drives:** Hard Disk Drives: Run-Length Limited (RLL) encoding is used in magnetic storage to improve data density and error correction.
- **Game Assets:** Game Assets: Simple sprites and tile maps with large uniform colour areas are often RLE-compressed to reduce ROM size in retro game systems.
- **Log File Compression:** Log File Compression: Server log files with long sequences of repeated status codes or timestamps can be pre-processed with RLE before archiving.

References

Textbooks:

1 Kernighan, B. W. and Ritchie, D. M. (1988). The C Programming Language, 2nd Edition. Prentice Hall.
2 Balagurusamy, E. (2019). Programming in ANSI C, 8th Edition. McGraw-Hill Education.
3 Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. (2009). Introduction to Algorithms, 3rd Edition. MIT Press.
4 Salomon, D. (2007). Data Compression: The Complete Reference, 4th Edition. Springer-Verlag.

Online Resources and Documentation:

1 GNU GCC Documentation. (2024). GCC, the GNU Compiler Collection. Available at: https://gcc.gnu.org/onlinedocs/
2 cppreference.com. (2024). C Standard Library Reference – <stdio.h>. Available at: https://en.cppreference.com/w/c/io
3 Wikipedia. (2024). Run-Length Encoding. Available at: https://en.wikipedia.org/wiki/Run-length_encoding
4 ITU-T Recommendation T.4. (2003). Standardization of Group 3 facsimile terminals for document transmission. International Telecommunication Union.